República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Guanta, Edo – Anzoátegui

Universidad Politécnica Territorial José Antonio Anzoátegui

Programación II

Fundamentos de POO

Profesor: Ing. Humner Lenis

Alumno: Jesús Acosta V-29.733.586

Programación Orientada a Objetos (POO)

Es un paradigma de programación que utiliza "objetos" para diseñar aplicaciones y programas de software. Se basa en modelar entidades del mundo real combinando datos (atributos) y comportamientos (métodos) en una sola estructura lógica.

Conceptos Fundamentales: Clases y Objetos

Antes de entrar en los pilares de la POO, es esencial entender sus dos bloques de construcción básicos:

- **Clase:** Es la plantilla, el plano o el molde a partir del cual se crean los objetos. Define qué atributos y métodos tendrá cualquier objeto creado a partir de ella.
- **Objeto:** Es una instancia concreta de una clase. Mientras que la clase Auto es un concepto abstracto, tu auto estacionado afuera es un objeto real con color, marca y velocidad específica.
- **Atributos:** Las variables que pertenecen a una clase y almacenan el estado del objeto (ej. color, peso, encendido).
- **Métodos:** Las funciones definidas dentro de una clase que describen los comportamientos o acciones que el objeto puede realizar (ej. acelerar(), frenar()).

Los 4 Pilares de la POO

La POO se sostiene sobre cuatro principios fundamentales que garantizan la modularidad, reutilización y mantenimiento del código:

1. Abstracción

Consiste en aislar los elementos de un contexto o enfocarse en las características esenciales de un objeto, ignorando los detalles secundarios o irrelevantes para el problema actual.

- Ejemplo: Si estás diseñando un sistema de gestión escolar para el departamento administrativo, te interesa el nombre, la cédula y las notas de un estudiante, pero no su tipo de sangre o su estatura.

2. Encapsulamiento

Es el mecanismo que agrupa los datos y los métodos que operan sobre esos datos dentro de una misma estructura (la clase), y **restringe el acceso directo** al estado interno desde fuera mediante modificadores de acceso (public, private, protected).

- Beneficio: Protege la integridad de los datos evitando que código externo modifique variables críticas de forma indebida. Para interactuar con los datos privados se utilizan métodos específicos conocidos como getters y setters.

3. Herencia

Permite que una nueva clase (llamada clase hija o subclase) herede los atributos y métodos de una clase existente (clase padre o superclase).

- Beneficio: Fomenta la reutilización del código. Si tienes una clase general Usuario con atributos como nombre y correo, puedes crear clases hijas como Administrador y Cliente que hereden esos campos sin necesidad de reescribirlos.

4. Polimorfismo

Literalmente significa "muchas formas". Es la capacidad de que diferentes clases respondan de manera distinta ante una misma llamada a un método.

- Ejemplo: Si tienes una clase padre Figura con un método calcularArea(), las clases hijas Circulo y Rectangulo pueden implementar ese mismo método con fórmulas matemáticas completamente diferentes adaptadas a su propia geometría.

UML (Diagramas)

UML, son las siglas en inglés de Unified Modeling Language (Lenguaje Unificado de Modelado). Es un lenguaje visual estándar que se utiliza en la ingeniería de software para visualizar, especificar, construir y documentar los componentes de un sistema orientado a objetos antes (o durante) el proceso de programación.

Así como un arquitecto no levanta un edificio sin antes dibujar los planos estructurales, los planos eléctricos y los diagramas de tuberías, un desarrollador de software utiliza UML para diseñar la estructura y el comportamiento de una aplicación antes de escribir miles de líneas de código.

¿Para qué sirve?

1. **Planificación y Diseño:** Te permite estructurar cómo funcionará el sistema, qué clases existirán, cómo se relacionarán entre sí y qué procesos ejecutará.
2. **Comunicación:** Facilita que todo el equipo (desarrolladores, analistas, diseñadores y clientes) hable el mismo idioma técnico sin confusiones.
3. **Documentación:** Deja un registro visual claro de cómo está construido el software, lo que facilita enormemente su mantenimiento o futuras actualizaciones.

Los Tipos de Diagramas UML más Utilizados

UML divide los diagramas en dos grandes grupos: estructurales (cómo es el sistema) y de comportamiento (cómo actúa el sistema). Entre los más importantes se encuentran:

1. Diagrama de Clases (Estructural)

Es el rey indiscutible de la POO. Muestra las clases que componen el sistema, sus atributos, sus métodos y las relaciones que existen entre ellas (por ejemplo, qué clase hereda de otra o qué clase usa a otra).

2. Diagrama de Casos de Uso (De Comportamiento)

Muestra las funcionalidades del sistema desde el punto de vista del usuario (o "actor"). Sirve para entender quién interactúa con el sistema y qué puede hacer (por ejemplo: un cliente puede iniciar sesión, buscar productos y realizar un pago).

3. Diagrama de Secuencia (De Comportamiento)

Muestra cómo interactúan los objetos entre sí a lo largo del tiempo, detallando el orden cronológico de los mensajes o llamadas a métodos que se envían unos a otros durante un proceso específico.

Especificadoras

En la Programación Orientada a Objetos (POO), cuando te refieres a especificadoras, estás hablando de los modificadores de acceso (también conocidos comúnmente como especificadores de acceso).

Estas palabras clave definen la visibilidad y el nivel de acceso que tienen los atributos y métodos de una clase. Es decir, controlan desde dónde se pueden ver, leer o modificar los datos de un objeto.

Los 3 Principales Modificadores de Acceso

1. public (Público)

- **Qué significa:** Lo que sea público puede ser accedido y modificado desde cualquier lugar, ya sea desde dentro de la misma clase, desde clases hijas, o desde cualquier otra parte del programa (por ejemplo, desde el programa principal o al instanciar el objeto).
- **Cuándo se usa:** Generalmente para los métodos que forman la interfaz pública de la clase (las acciones que el usuario o el sistema necesitan invocar libremente).

2. private (Privado)

- **Qué significa:** Es el nivel más restrictivo. Los elementos privados solo pueden ser accedidos desde el interior de la propia clase donde fueron declarados. Ninguna clase externa ni ninguna clase hija puede verlos ni tocarlos directamente.
- **Cuándo se usa:** Es la base del encapsulamiento. Se usa para proteger los datos sensibles del objeto (como la contraseña de un usuario, el saldo de una cuenta bancaria, etc.) para evitar que sufran alteraciones accidentales o malintencionadas. Para consultar o modificar estos datos de forma segura se usan los métodos getters y setters.

3. protected (Protegido)

- **Qué significa:** Tiene un comportamiento intermedio. Los elementos protegidos se pueden ver desde la propia clase y también desde sus clases hijas (en la herencia), pero siguen estando ocultos para el resto del mundo exterior (clases independientes).
- **Cuándo se usa:** Cuando estás diseñando una jerarquía de clases y quieres que las clases descendientes (hijas) tengan acceso directo a ciertos atributos o métodos del padre, sin abrirlos al público general.

Intefaces

Una interfaz (Interface) en la Programación Orientada a Objetos es como un contrato o un estándar de diseño.

Define qué acciones debe realizar obligatoriamente una clase, pero no cómo debe realizarlas (es decir, declara los métodos sin escribir su código interno o cuerpo).

La analogía del mundo real

Piensa en un enchufe de pared. El estándar (la interfaz) define que cualquier dispositivo con una clavija de dos patas redondas podrá conectarse a la corriente para recibir energía.

- A la corriente no le importa si el dispositivo es una lámpara, un ventilador o un cargador de celular; todos firman el "contrato" de tener la clavija correcta.
- Cada aparato funciona por dentro de manera totalmente distinta, pero todos comparten la misma interfaz externa para interactuar.

Características Clave de las Interfaces

1. **Es un contrato estricto**: Si una clase decide "implementar" una interfaz, está obligada por ley del compilador a escribir el código de todos los métodos que esa interfaz declare. Si le falta uno solo, el programa dará un error.

2. **Solo define el qué, no el cómo:** Tradicionalmente, una interfaz solo contiene las firmas de los métodos (su nombre y parámetros), totalmente vacíos de lógica.
3. **Permite múltiples interfaces:** A diferencia de la herencia tradicional (donde en muchos lenguajes una clase solo puede tener un padre), una sola clase puede implementar varias interfaces al mismo tiempo.